Approximate Skyline Index for Constrained Shortest Pathfinding with Theoretical Guarantee

Ziyi Liu[1], Lei Li[2,1,*], Mengxuan Zhang[2], Wen Hua[3], Xiaofang Zhou[1,2]
[1]The Hong Kong University of Science and Technology, Hong Kong SAR, China
[2]DSA Thrust, The Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China
[3]The Hong Kong Polytechnic University, Hong Kong SAR, China
liuziyi30@gmail.com, thorli@ust.hk, mengxuanz@hkust-gz.edu.cn, wency.hua@polyu.edu.hk, zxf@cse.ust.hk

2024 IEEE 40th International Conference on Data Engineering (ICDE) | 979-8-3503-1715-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICDE60146.2024.00322

***Abstract*—The *Constrained Shortest Path (CSP)* problem seeks to identify the shortest path between two vertices in a road network while adhering to a specific constraint on another criterion. Solving the *CSP* problem frequently entails navigating the two-criteria skyline path problem, which incurs a substantial computational expense in large road networks. The primary challenge lies in handling a vast quantity of partial skyline paths, which often hinders index-based solutions from accurately determining the skyline paths. This paper introduces α-*FHL*, a practical approximation method designed to circumvent the costly skyline path search and hasten computation on skyline path indexing. α-*FHL* uses *tree decomposition* to hierarchically assign approximation ratios, thereby facilitating effective pruning within the labelling index. Moreover, we devise various strategies to allocate approximation ratios and an efficient approximation concatenation method to respond to the approximate *CSP* queries via the α-*FHL* index. Our method culminates in swift index construction and efficient query response. Comprehensive experiments conducted on real-world road networks substantiate the superiority of our approach over contemporary solutions**

I. Introduction

Route planning is integral to our daily routines, and various path optimization challenges have been explored in recent years. Existing solutions primarily focus on a single objective, such as minimizing distance [1]–[3], travel time [4]–[8], fuel consumption [9], [10], battery usage [11], or diversifying routes [12]–[15], among others. However, real-world route planning often requires accounting for various auxiliary criteria to meet user preferences, such as tunnel height, road capacity, slope gradient, total elevation gain, duration of tourist drives, or the number of transportation changes. For instance, a user may need to limit the budget for tolls on highways, bridges, or tunnels; in populous cities, drivers often aim to make fewer left turns (assuming right-side driving) to avoid potential accidents. Therefore, traditional shortest path algorithms may not provide optimal solutions when these auxiliary criteria are considered. As a response, the *Constrained Shortest Path (CSP)* method was developed. It identifies the optimal path based on a primary objective while adhering to a set of predefined constraints on other criteria. For example, with two objectives (cost c and distance d) and a maximum constraint C on c, the *CSP* solution selects the shortest path p with cost $c(p)$ less than C. Despite the absence of an upper limit on the number of possible criteria [16]–[18], our study concentrates on the single-criterion *CSP* problem, which is significant due to the following reasons: 1) It establishes the foundation for the multi-criteria *CSP*, and even within this foundational stage, addressing the single-criterion *CSP* query presents inherent challenges; 2) Excessively combined criteria may render the routing system less usable, as the algorithm might not return any viable path; 3) By using a single-criterion *CSP*, users can formulate their queries more straightforwardly, minimizing the potential for conflicting constraints or ambiguities. This ensures a more accurate interpretation of user intent by *CSP* methods, leading to more satisfactory results.

Existing Solutions. Contemporary approaches to the queries broadly belong to two categories: exact and approximate solutions. The exact *CSP* algorithms provide precise answers that fully meet the user's requirements. However, their computation is notably time-intensive, given their status as NP-Hard problems [19]. Moreover, they inherit the complexities of the skyline path search [20]–[22], which generates a set of paths, none of which dominate others across all criteria.

In index-free *CSP* algorithms, every vertex visited retains a set of skyline paths originating from the source. The challenge lies in determining which paths should be retained to compose the final skyline path for subsequent searches. This process results in significant memory usage, which is why an exact index-free solution struggles to process *CSP* queries efficiently on real-world road networks. While exact index-based methodologies have been proposed, they present greater challenges, as balancing query efficiency with index size is nontrivial. For example, *CSP-CH* [23] preserves only a few skyline results on its shortcuts to maintain construction and index size, but this approach negatively impacts its query time. *Forest hop labeling (FHL)*-based methods [17], [18], [24] offer the quickest query processing by eliminating online searches with tree decomposition. However, its index is considerably larger than other index-based methods due to the need to store a set of exact skyline indices as 2-hop labels for each vertex.

Conversely, approximate *CSP* algorithms compromise on path length optimality to conserve computational time and indexing space. In particular, these solutions predefine an approximation ratio α, ensuring the length of the final path returned does not exceed α times the length of the optimal path. Most index-free approximate methods offer limited

* Lei Li is the corresponding author.

2375-026X/24/$31.00 ©2024 IEEE
DOI 10.1109/ICDE60146.2024.00322

speedup compared to exact solutions because they still contend with high computational costs in large road networks. Currently, no approximate solution has been directly applied to the *CSP* index. *COLA* [25] represents the cutting-edge in approximate solutions. However, its performance enhancement is constrained as the approximation is applied to search-based query processing rather than the index.

Non-Necessity Cases of Exact Methods. In real-world scenarios, the necessity for the exact *CSP* is limited. Firstly, the user's specific requirements can often be met with a good approximation rather than the exact shortest path. Consider a driver who wishes to minimize the travel distance while reducing toll charges. An approximate path that slightly extends the travel distance while effectively avoiding more toll roads can still satisfy the user's needs. Secondly, the exactness of a route can be compromised by the unpredictable nature of traffic, road conditions, and other dynamic factors. These uncertainties can transform an optimal route into a non-optimal one after the plan has been made, indicating that a near-optimal solution may suffice in many practical cases. Thirdly, the computational resources required to calculate the exact CSP can be prohibitive, especially when the constraints are complex or the network is extensive. Hence, an approximate CSP solution, which can provide near-optimal results with significantly less computational cost, is not only practical but can be more efficient for many applications.

Challenges and Contributions. In our quest for an efficient, fast-to-construct approximate *CSP* index, we propose the α-*FHL* index. This work builds upon the foundations set by [24], [17], and [18], which effectively integrate 2-hop-based skyline path concatenation for query processing and tree decomposition-based indexing. [24] investigates *CSP* query responses under two criteria, with a focus on techniques for constructing the 2-dimensional skyline index. The study capitalizes on the inherent properties of 2-dimensional skyline paths to facilitate their concatenation. Simultaneously, [18] expands on [24] to include multi-constraints shortest path queries. The focus is mainly on optimizing the efficiency of the multi-dimensional skyline path index. This optimization becomes imperative as the time complexity of skyline computation increases significantly with the introduction of multiple constraints. Moreover, the natural properties utilized in [24] offer limited optimization benefits in multi-constraints scenarios. On another note, [17] focuses on providing a more user-friendly querying approach that supports any combination of constraints in queries. The resulting index, termed skyline path cubes, organizes skyline paths across different dimensions. In this paper, we aim to build upon these previous efforts by approximating the skyline path index through the exploration of approximation ratio allocation on tree decomposition. Our goal is to boost the performance of indexing, streamline query processing, and minimize memory consumption. However, actualizing these goals presents substantial challenges, which we detail in subsequent sections.

Approximate Framework. Let F represent the solution for constructing the *FHL* index and Λ° symbolize the index result of graph G. Given the approximation ratio $\alpha \geq 1$ as a parameter of F, the optimal approximation result of $F(\alpha, G)$ would prune Λ° to $\alpha \otimes \Lambda^\circ$ such that $F(\alpha, G)$=$\alpha \otimes \Lambda^\circ \subseteq \Lambda^\circ$ ($\otimes$ indicates Λ° approximated by α to achieve the minimum approximated index). In $\alpha \otimes \Lambda^\circ$, no two skyline paths can α-dominate each other. A special case arises when α=1, such that $F(1, G)$=$1 \otimes \Lambda^\circ$=Λ°, which signifies the exact *FHL* [24] index. However, $\alpha \otimes \Lambda^\circ$ means that α is acting on Λ°, thus suggesting the exact index is obtained first and then pruned by α. This process is expensive and more time-consuming than the exact *FHL*. As such, we propose an approximate *FHL* framework that prunes the index at different indexing phases. Specifically, the *FHL* indexing consists of two phases: *skyline tree decomposition* and *skyline label assignment*. In our framework, we design a bottom-up approximate tree decomposition to form trees with approximate skyline shortcuts, and a top-down approximate label assignment strategy. Notably, our approximation techniques revolve around the structure of tree decomposition, a subject yet to be thoroughly investigated.

Allocation of α. Allocating α for pruning during *FHL* index computation is challenging for several reasons: 1)Directly using α to calculate the approximate index leads to an over-approximation ratio in the results, yielding far from optimal *CSP* query answers; 2)A small approximation ratio applied at each iteration results in insufficient pruning power, generating a less effective and redundant index, thus impacting query processing efficiency; 3)Maintaining the same pruning power is difficult as each label may experience different approximation times; 4)A fixed allocation plan is inadequate to solve the problem. If we compute a label with high approximation, subsequent labels will be approximated insufficiently. Hence, we propose efficient methods to allocate α and recycle overused approximate power. Moreover, we theoretically analyze the relationship between α allocation and the tree decomposition structure, which can provide support for the approximate scheme of tree decomposition-based indices.

Skyline Operator. Skyline path concatenation, a crucial operator in both index construction and query processing, is time-consuming, particularly in large-scale graphs. To compute a label $L(v, u)$, m skyline paths in $L(v, w_i)$ and n skyline paths in $L(w_i, u)$, where w_i is a hop between v and u, must be concatenated. For multiple intermediate hops, the concatenated paths through each must be computed. Assume that we generate a set P_s including all concatenated results and select the final approximate skyline paths of P_s. Given that the size of hops is h, we will concatenate around mnh times. Although α-*FHL* can reduce concatenation times by approximating each label, dominance verification is still required in both single-hop and multiple-hop concatenations. This is because, we cannot guarantee the concatenated results from v to u remain skyline paths, requiring $O(mn\log(mn))$ time to verify their approximate dominance relations. To expedite this process, we designed an efficient approximate algorithm α-SPC_1 for single-hop concatenation and a pruning technique α-SPC_n to reduce the number of intermediate hops. Our contributions are summarized as follows:

- We offer an approximate framework for *FHL* with a theoretical guarantee of our approximation's correctness.
- We suggest a threshold to selectively approximate the indexing labels and a recycling technique to reuse redundant approximate power on labels.
- We introduce an efficient operator to concatenate skyline paths obtained from different approximation ratios, enabling quick discovery of concatenated skyline results approximated by a specific ratio.
- We provide a comprehensive evaluation of our methods using extensive real-life road networks, showcasing superior performance over current methods in terms of query time, index size, and construction time.

II. Preliminary

A road network is an undirected graph $G(V, E)$, where V denotes a set of vertices, and $E \subseteq V \times V$ signifies a set of edges. Each edge $e \in E$ is characterized by two metrics: weight $w(e)$ and cost $c(e)$. A path p extending from the source $s \in V$ to the target $t \in V$ is a sequence of consecutive vertices. Each path p has an associated weight $w(p)$ and a cost $c(p)$. Our work utilizes an undirected road network to simplify descriptions and lay emphasis on relations between approximation ratio and tree structure. Additionally, it can be easily extended to a directed version as describe in Section IX. We initially define an *CSP* query as follows:

Definition 1 (**CSP Query**). *Given a graph G, a source s, a destination t, and a maximum cost constraint $C \in \mathbb{R}^+$, a CSP query $q(s, t, C)$ returns a path $\hat{p}$ with minimum $w(\hat{p})$ while guaranteeing $c(\hat{p}) \leq C$.*

Next, we introduce the concept of an α-*CSP query*, which serves as an approximation to the *CSP* query.

Definition 2 (α-**CSP Query**). *Given a graph G, a source s, a destination t, a maximum cost bound $C \in \mathbb{R}^+$, and an approximation ratio $\alpha \in [1, 2]$, an α-CSP query $q(s, t, C, \alpha)$ returns a path p with the weight $w(p) \leq \alpha \cdot w(\hat{p})$ while guaranteeing that $c(p) \leq C$, where $\hat{p}$ is the result of the exact CSP query $q(s, t, C)$.*

Problem Definition. Given a graph G, our goal is to construct an index L capable of efficiently processing any α-*CSP* query $q(s, t, C, \alpha)$.

Unlike the index for the shortest path queries, which only stores the shortest distance from one vertex to another, the α-*CSP* index stores skyline paths to support any constraint C. In addition, the *CSP* paths are essentially a subset of the *skyline* paths, and the α-*CSP* paths are a subset of the *CSP* paths.

A. Skyline Path

Initially, we establish the *dominance* relation between any two paths with identical sources and destinations:

Definition 3 (α-**Path Dominance**). *Given two paths p_1 and p_2 with the same s and t, p_1 α-dominates p_2 iff $w(p_1) < \alpha \cdot w(p_2)$ and $c(p_1) \leq c(p_2)$, or $w(p_1) \leq \alpha \cdot w(p_2)$ and $c(p_1) < c(p_2)$.*

Definition 4 (**Skyline Path**). *Given any paths p_1 and p_2 with the same s and t, p_1 and p_2 are skyline paths if they cannot α-dominate each other.*

Hereafter, we employ α_i-skyline path to emphasize the skyline paths that cannot α_i-dominate each other.

Definition 5 (**Skyline Path Set**). *In a set $P(s, t)$ of paths from s to t, we can obtain a set $P_s(s, t) \subseteq P(s, t)$ of skyline paths based on the definition 3. P_s is maximal when α=1.*

Consider that we can set α=1 to generate the complete skyline index for *CSP* query processing.

Example 1. *In Figure 1-(a), each edge contains two criteria (w, c). When α=1, we find three skyline paths from v_3 to v_7: p_1=$\langle v_3, v_2, v_7\rangle$=$(2, 8)$; p_2=$\langle v_3, v_1, v_2, v_7\rangle$=$(3, 7)$; p_3=$\langle v_3, v_1, v_7\rangle$=$(6, 3)$. The other paths from v_3 to v_7 are dominated, e.g., p_4=$\langle v_3, v_2, v_1, v_7\rangle$=$(7, 6)$ is dominated by p_3. Once we increase α to 1.5, p_1 is dominated by p_2, because $w(p_2)$=$\alpha \cdot w(p_1)$=3 and $c(p_2) < c(p_1)$. Thus, the skyline paths from v_3 to v_7 remain p_2 and p_3.*

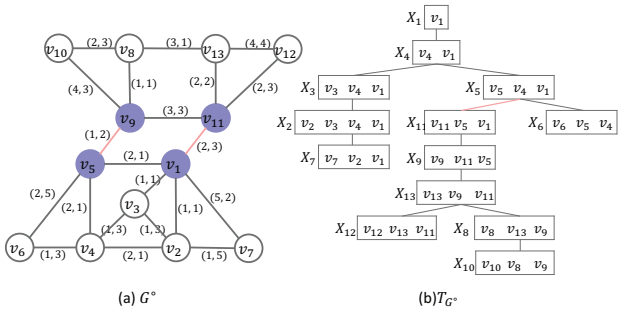


Fig. 1. Example network G° with its tree decomposition T_{G°

Theorem 1 (**Non-empty skyline path set**). *For any $\alpha \geq 1$, if two vertices s and t are reachable in graph G, the cardinality of the set $P_s(s, t)$ is at least one, i.e., $|P_s(s, t)| \geq 1$.*

Proof. When vertices s and t are reachable, $|P(s, t)| \geq 1$. If $|P(s, t)|$=1, the sole path in $P(s, t)$ becomes a skyline path in $P_s(s, t)$. If $|P(s, t)| > 1$, any path p with the minimum cost in $P(s, t)$ cannot be α-dominated by other paths, given that α applies to $w(p)$. So p belongs to $P_s(s, t)$ for any $\alpha \geq 1$. □

Theorem 2 (**Completeness of Skyline Index**). *The skyline paths between any pair of vertices are both complete and minimal for every conceivable α-CSP query.*

Proof. The completeness of the skyline index can be established in a manner similar to that presented in [24]. □

Therefore, constructing an index for an α-*CSP* query is equivalent to creating the index for a skyline path query.

B. Forest Hop Labeling (FHL) Index

The *FHL* data structure is derived from the underlying tree decomposition, as defined below.

Definition 6 (**Tree Decomposition**). *Given a graph $G(V, E)$, its tree decomposition, denoted as T_G, is a rooted tree. In this tree, each node X_i is a subset of the vertex set V. The tree possesses the following characteristics: 1)The union of all X_i equals V; 2)For every edge $(u, v) \in E$, there exists an X_i such that $u, v \subseteq X_i$; 3)For each vertex $v \in V$, the set $\{X_i | v \in X_i\}$ forms a subtree of T_G.*

Definition 7 (**Tree Width and Tree Height**). *Let the maximum size of all nodes in T_G be $max_{X \in T_G}|X|$. The tree width of T_G, denoted by $\mathcal{W}_G$, is computed as W_G=$max_{X \in T_G}|X| - 1$. The tree height of T_G, denoted by $\mathcal{H}_G$, is the maximum depth of all nodes in T_G.*

For the sake of brevity, we use $\mathcal{W}_v$ and $\mathcal{H}_v$ to denote the tree width and tree height of X_v in T_G,

Example 2. *In T_{G° of Figure 1-(b), $\mathcal{H}_{G^\circ}$=8, $\mathcal{W}_{v_2}$=3, $\mathcal{H}_{v_2}$=4.*

The *FHL* index is specifically designed to answer *CSP* queries. Thus, α is set to one by default. To mitigate the large index size introduced by lengthy skyline paths, *FHL* initially partitions G into a set $\{G_1, \ldots, G_k\}$ of vertex-disjoint subgraphs such that $\bigcup_{i \in [1,k]} V(G_i)$=$V(G)$, and $V(G_i) \cap V(G_j)$=$\emptyset$ for all $i \neq j, i, j \in [1, k]$. An edge connecting two subgraphs is referred to as a *boundary edge*, like the edges (v_5, v_9) and (v_1, v_{11}) in Figure 1-(a). These edges divide the example graph into two partitions: G_1° comprising $\{v_1, v_2, v_3, v_4, v_5, v_6, v_7\}$ and G_2° containing $\{v_8, v_9, v_{10}, v_{11}, v_{12}, v_{13}\}$. The terminal vertices of a boundary edge are termed *boundary vertices*.

Moreover, *FHL* extracts all boundary edges and their associated vertices to form a *boundary graph*. Importantly, *FHL* precomputes the skylines between every pair of boundary vertices in each partition to ensure correct index construction, adding shortcuts for them if they are not connected in the original graph. Utilizing tree decomposition, we map each subgraph onto a small tree structure and the boundary graph onto a boundary tree. The small trees and the boundary tree together constitute a forest.

Based on the tree decomposition of each subgraph and a boundary graph, *FHL* computes the labels as defined below.

Definition 8 (**Forest Hop Labeling (FHL)**). *For each subgraph and a boundary graph $G_i(V_i, E_i) \subseteq G$, FHL computes the skyline label $L(v)$ for every $v \in V_i$. The label $L(v)$ stores the skyline paths from v to each of its ancestors on T_{G_i}, and $L(v, a_i)$ denotes the skyline paths from v to its ancestor a_i; that is, $L(v, a_i)$=$P_s(v, a_i)$.*

1) FHL Indexing: The construction of the index on each subgraph and a boundary graph, as detailed in Algorithm 1, comprises two main phases: a bottom-up Skyline Tree Decomposition, which collects the skyline shortcuts, generates the tree nodes, and builds the tree, and a top-down Skyline Label Assignment, which creates the labels necessary for query answering.

We briefly introduce the two phases shown in Algorithm 1.

Phase1: Skyline Tree Decomposition. The construction of the corresponding tree nodes for each vertex takes place in this phase, involving the contraction of vertices in a particular sequence (lines 1-5), which is obtioned from the *minimum degree elimination* [21], [26]. Once a vertex v is contracted, its vertex set X_v includes v and its neighbour set $N(v)$. This contraction also generates skyline shortcuts $P_s(u, w)$ between each pair of neighbours (u, w) in $N(v)$, accomplished by concatenating skyline paths from v to u and those from u to w. Note that the computation of $P_s(u, w)$ can be skipped for boundary vertices as their shortcuts are precomputed. After creating all the tree nodes, we form a tree by assigning the parent of each X_v as X_u, where $u \in X_v$ is the vertex with the lowest rank in X_v (lines 6-7).

Phase2: Skyline Label Assignment. This phase involves the population of labels in a depth-first approach from the tree root, employing the labelling of its ancestors (lines 8-13). Specifically, we assign labels from X_v to all its ancestors X_u to v since the vertices in X_v naturally establish the *cut property* between v and all its ancestors.

Algorithm 1: *FHL* Index Construction

```
   Input: Graph G(V, E)
   Output: CSP-2Hop Labeling L
1  while v ∈ V has the minimum degree and V ≠ φ do
2      X_v=N(v), r(v) ← Iteration Number            ▷ Tree Node Contraction
3      for (u, w) ← N(v) × N(v) do
4          P_s(u, w) ← Skyline(P_s(u, v) ⊕ P_s(v, u), (u, w));
5          E ← E ∪ (P_s(u, w)), V ← V − v,
             E ← E − (u, v) − (v, w);
6  for v in r(v) increasing order do
7      u ← min{r(u)|u ∈ X(v)}, X_v.Parent ← X_u   ▷ Tree Formation
8  X_v ← Tree Root, Q.insert(X_v)                  ▷ Label Assignment
9  while X_v ← Q.pop() do
10     for u ∈ {u|X_u is ancestor of X_v} do
11         P_s(v, u)) ← Skyline(P_s(v, w) ⊕ P_s(w, u)), ∀w ∈ X_v;
12         L(v) ← (u, P_s(v, u));
13     Q.insert(X_u.children), L ← L ∪ L(v);
```

2) FHL Querying: The structure of *FHL* comprises a *cut property*, which is utilized to address queries. For $\forall s, t \in V$, where X_s and X_t represent their corresponding tree nodes without an ancestor/descendant relation, their cuts are found in the *lowest common ancestor (LCA)* node [27].

For queries with s and t on different small trees, the *CSP* results are derived with the assistance of the boundary tree.

Considering a *CSP* query q=(s, t, C), in the case where s and t belong to the same tree, the first step is to determine the common intermediate hop set H=$\{h | h \in LCA(s, t)\}$ by extracting the vertices in the *LCA* of s and t, and $|H| \leq W_G$. The term h is specifically employed to denote the vertices in the *LCA*, so that $P_c(s, h, t)$ symbolizes the skyline path candidates extending from s to t via h. Following this, $\forall h_i \in H$, the computation of skyline path candidates $P_c(s, h_i, t)$ is performed by executing $P_s(s, h_i) \oplus P_s(h_i, t)$. The set of candidates $P_c(s, t)$ is obtained by consolidating all $P_c(s, h_i, t)$ with the use of the formula:

$$\begin{aligned} P_c(s,t) &= \{P_c(s,h_i,t) | \{P_s(s,h_i) \oplus P_s(h_i,t)\}\}, \forall h_i \in H \\ &= \{\{p_j(w_j(s,h_i) + w_k(h_i,t), c_j(s,h_i) + c_k(h_i,t))\}\} \\ &\quad \forall p_j \in P_s(s,h_i) \wedge p_k \in P_s(h_i,t) \end{aligned} \quad (1)$$

Here, each $P_c(s, h_i, t)$ includes all the pairwise skyline concatenation results.

If $s \in T_{G_1}$ and $t \in T_{G_2}$, let H_1 and H_2 be the boundary vertices in $T_{G_1}$and T_{G_2}, $P_c(s,t)$ is calculated in three steps:

Step 1: For each $h_1^i \in H_1$, we compute $P_s(s, h_1^i)$. The computation aligns with Equation 1 because both s and h_1^i belong to the same tree.

Step 2: For each $h_2^j \in H_2$, we compute $P_c(s, h_2^j)$ including all skyline candidates via $\forall h_1^i \in H_1$ by the equation:

$$P_c(s, h_2^j)=\{P_c(s, h_1^i, h_2^j)|\{P_s(s, h_1^i) \oplus P_s(h_1^i, h_2^j)\}\} \quad (2)$$

where $P_s(s, h_1^i)$=$L(s, h_1^i)$ is stored in T_{G_1}, and $P_s(h_1^i, h_2^j)$ is computed using the labels in T_{G_b} since both h_1^i and h_2^j are boundary vertices

Step 3: The calculation of $P_c(s,t)$ involves the union of all $P_c(s, h_2^j, t)$ as follows:

$$P_c(s,t)=\{P_c(s, h_2^j, t)|\{P_c(s, h_2^j) \oplus P_s(h_2^j, t)\}\} \quad (3)$$

where $\forall h_2^j \in H_2$, each $P_c(s, h_2^j)$ is already computed in Step 2, and $P_s(h_2^j, t)$=$L(h_2^j, t)$ is recorded in T_{G_2}.

Ultimately, the optimal path that meets the constraint C of the *CSP* query q=(s,t,C) is found in $P_c(s,t)$.

Remark. The *FHL* index for $\forall v \in V$ includes a set $P_s(v, a_i)$ of skyline paths from v to a_i, with a_i denoting each ancestor of v in T_G. $P_s(v, a_i)$ is considered a label $L(v, a_i)$ stored in $L(v)$. The comprehensiveness of the skyline path index restricts the scalability of *FHL*, as the information about the skyline path can be extensive in dense or large-scale networks. It's inevitable that *FHL* generates a larger volume of labels, particularly for vertices situated far from tree root.

III. α-*FHL* CONCATENATION

The α-*FHL* utilizes a hop-based indexing method, which designates approximate labels to each vertex within G. In contrast with the *FHL*, α-*FHL* prunes each label that has an approximation ratio exceeding 1. This section introduces α-*skyline path concatenation (α-SPC)*, a fundamental operation to build an α-*FHL* index and to address α-*CSP* queries.

Given P_α, which symbolizes a set of α-skyline paths, $P_{\alpha_1}(s,t)$ denotes a set of α_1-skyline paths that originates at s and terminates at t. Here, we first establish a complete P_α.

Definition 9 (Complete α-Skyline Path Set). *$P_s(u,v)$ comprises all exact skyline paths from s to t, $P_\alpha(u,v)$ is complete if it has the maximum count of α-skyline paths derived from $P_s(u,v)$. We denote the complete $P_\alpha(u,v)$ as $\mathcal{P}_\alpha(u,v)$.*

Definition 10 (α-Skyline Path Concatenation (α-SPC)). *The concatenation result $\mathcal{P}_\alpha(u,v)$ is calculated by two operators: 1)α-SPC_1 computes $\mathcal{P}_{\alpha_0}(u, h_i, v)$=$\mathcal{P}_{\alpha_1}(u, h_i) \oplus \mathcal{P}_{\alpha_2}(h_i, v)$, $\forall h_i \in H'$, where H' is a set of screened hops between u and v. 2)α-SPC_n prunes the hops in H, which encompass all intermediate hops, and forms $H' \subseteq H$ to direct α-SPC_1. It then forms the union $P_\alpha(u,v)$=$\bigcup_{h_i \in H'} \mathcal{P}_\alpha(u, h_i, v)$, and retrieves $\mathcal{P}_\alpha(u,v)$ from $P_\alpha(u,v)$.*

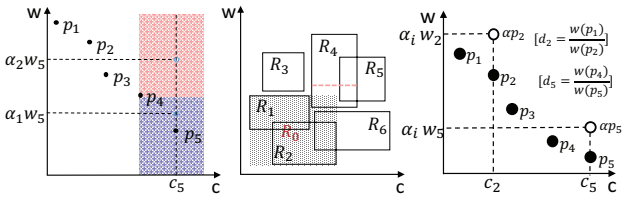
Observe that α-SPC_1 operates on a single hop, while α-SPC_n emphasizes pruning across multiple hops. Henceforth, we notate $\mathcal{P}_\alpha(u,v)$ as $\mathcal{P}_\alpha$ and $\mathcal{P}_\alpha(u, h_i, v)$ as $\mathcal{P}_\alpha(h_i)$, given the context clarity.

A. *Operator α-SPC_1*

Within α-SPC_1, the operator $\oplus$ is similar to the skyline path concatenation of *FHL*. We first compute the weight and cost of a set P_c of concatenated paths and then confirm their α-dominance. However, verifying $\mathcal{P}_\alpha$ from P_c differs significantly, as a correct $\mathcal{P}_\alpha$ heavily depends on the order of verification in P_s. For exact skyline paths, we can promptly discard a path p in P_c once another path p' dominates p. However, this method potentially leads to inaccuracies when deriving $\mathcal{P}_\alpha$, as shown in the following example:

Example 3. *In P_c=$\{(6,14),(7,12),(8,12),(9,10)\}$, the exact skyline path P_s can be obtained by comparing each pair by weight-increasing order and discarding dominated ones, resulting in P_s=$\{(6,14),(7,12),(9,10)\}$. However, with $\alpha = 1.35$, $(6,14)$ is α-dominated by $(7,12)$, and $(8,12)$ is α-dominated by $(9,10)$, leaving only $(9,10)$ in $\mathcal{P}_\alpha$. But, the correct $\mathcal{P}_\alpha$ is $\{(6,14),(9,10)\}$.*

Therefore, α-SPC_1 utilizes a greedy concatenation method to generate and verify concatenated paths in a cost-increasing order (i.e., the criterion without α). This guarantees the correctness of $\mathcal{P}_\alpha$ due to two reasons: 1)It is ascertained that the first concatenated path with the minimum cost is an α-skyline path; 2)For the current concatenated path p_i (the subscript indicates the order of produced paths), it is solely required to verify if the current concatenated path can be α-dominated by the preceding α-concatenated skyline path. For instance, if p_{i-1} is confirmed as an α-skyline path, the dominance of p_i is verified by comparing $\alpha \times w(p_{i-1})$ with $w(p_i)$. If p_{i-1} fails to dominate p_i, all the skyline paths formulated before p_i are incapable of dominating p_i. Additionally, p_i cannot be dominated by subsequent concatenated paths due to their costs being higher than $c(p_i)$.



(a) Approximate Greedy Concatenation on the Current Concatenated Path (p_5)

(b) Example of Approximate Rectangle-Pruning

(c) Example of F_2-R ($d_5 < d_2 < \alpha_i$)

Fig. 2. α-*FHL* Pruning Techniques

Example 4 (α-SPC_1)**.** *As illustrated in Figure 2-(a), the current concatenated path is p_5, and paths from p_1 to p_4 have already been verified as α-skyline paths. These are plotted on a cost-weight axis. The domination areas of p_4 are demonstrated, where all points in the red area are dominated by p_4. When α>1, the domination of p_5 falls into two cases: 1) if α=α_1, it generates a point $(\alpha_1 w_5, c_5)$ located in the blue*

area, such that p_5 is an α_1-skyline path; 2) if α=α_2, where α_2>α_1, the point $(\alpha_2 w_5, c_5)$ is located in the dominating area (red) of p_4, such that p_5 is α_2-dominated by p_4.

B. Operator α-SPC_n

In G, there are likely many intermediate hops for an *origin-destination (OD)* pair (u, v). To enhance the computation of α-SPC, α-SPC_n incorporates an approximate rectangle-pruning method aimed at reducing the number of h_i to be considered. Essentially, this method identifies a range R_i of $\mathcal{P}_\alpha(h_i)$, and subsequently determines a range R_0 of $\mathcal{P}_\alpha$ before concatenating at each h_i. If any R_i is outside of R_0, the concatenation at h_i can be skipped.

The range of concatenated skyline paths $P_s(h_i)$ is determined by the following theorem:

Theorem 3 (**Endpoint Skyline**). *Given two labels $L(u, h_i)$ with n skylines and $L(h_i, v)$ with m skylines, both sorted in increasing weight order. Their two endpoint skyline paths are p_1, p_n and p'_1, p'_m, respectively. We denote p_1^1=$p_1 \oplus p'_1$ and p_n^m=$p_n \oplus p'_m$. Consequently, p_1^1 and p_n^m must be the skyline paths in the concatenated results of $L(u, h_i)$ and $L(h_i, v)$.*

For each hop h_i, p_1^1 is the first concatenated path and p_n^m is the last concatenated path. Among the paths via h_i, p_1^1 has the smallest weight with the largest cost, and P_n^m has the smallest cost with the largest weight. Thus, the mapping points of p_1^1 and p_n^m form a rectangular range R_i of concatenated results $P_s(h_i)$. Comparing all the endpoint skylines of the rectangles, we select the point with the smallest weight and the other with the smallest cost to form a range R_0, which can cover all the concatenated skyline paths in $L(u, v)$. Thus, we can prune all hops if their corresponding rectangles are outside R_0. Nevertheless, We can further narrow down the rectangles by applying an α. For a hop h_i, suppose that the weight of the last concatenated path through h_i is w_{min}, which provides the lower-bound of R_i, then we create a new lower-bound of dominance using $\alpha \times w_{min}$. Thus, the shrunken range has a higher chance of being further pruned by others.

Example 5 (α-SPC_n). *Figure 2-(b) illustrates the computation of $L(u, v)$ considering six hop vertices (h_1 to h_6) in the LCA of X_u and X_v. We generate six rectangles like R_3, representing the range of skyline paths from u to v via hop h_3. The path with the lowest cost maps to the upper-left endpoint and the one with the lowest weight to the bottom-right. Hops h_3 and h_5 are pruned as R_3 and R_5 lie outside R_0. R_1-R_4 are further scrutinized using α. The red line inside R_4 depicts its dominance lower-bound. As the entire shrunken range of R_4 falls within R_1's first quadrant, R_4 can be pruned, ensuring that α-skyline paths in R_1 consistently weight less and cost less than those in R_4.*

IV. α-FHL Framework

This section presents a theoretical analysis of the α-*FHL* framework. Let F be a solution to construct *FHL* index. Then, $F(\alpha, G)$ represents a function to build a *FHL* index Λ^α approximately, i.e., α-*FHL* indexing, on a graph G by α. For the specific case α=1, $F(1, G)$ returns the exact *FHL* index, which is denoted as Λ°. Thus, the naive method is $F(\alpha, G)$=$\alpha \otimes \Lambda^\circ$, which means that we obtain Λ^α by allocating α on each label of Λ°. Then, we define $F(\alpha, G)$in the following.

Definition 11 (α-***FHL* indexing**). *Given $\alpha_1, \alpha_2 \leq \alpha$ and a graph G, the function $F(\alpha, G)$ includes two nested functions: 1) $g(\alpha_1, G)$, an approximate bottom-to-up tree decomposition function that returns approximate skyline shortcuts μ^α based on the tree structure; 2) $f(\alpha_2, g(\alpha_1, G))$, an approximate top-to-down label assignment function that returns Λ^α.*

Therefore, $F(\alpha, G)$=Λ^α=$f(\alpha_2, g(\alpha_1, G))$, which constructs α-*FHL* by approximating either Λ°, function f, or function g. Our α-*FHL* is thus generalized by the following equation:

$$f(\alpha_2, g(\alpha_1, G))= \begin{cases} F_1(\alpha, G)=\alpha \otimes \Lambda^\circ & \text{if } \alpha_1=1, \alpha_2=1 \\ F_2(\alpha, G)=f(\alpha, g(1, G)) & \text{if } \alpha_1=1, \alpha_2=\alpha \\ F_3(\alpha, G)=f(1, g(\alpha, G)) & \text{if } \alpha_1=\alpha, \alpha_2=1 \end{cases} \quad (4)$$

One may question why we cannot establish a scenario where $\alpha_1\alpha_2$=α. This is because, dispersing α would compromise the pruning power in both phases, rendering it insufficient. Hence, we conceptualize F_1 as a baseline that provides valuable insights for analyzing the correctness of queries and defines a lower bound for the index. Both F_2 and F_3 illustrate the impact of approximation on different index phases. Furthermore, given that *skyline tree decomposition* is built upon *Contraction Hierarchies (CH)*, the design of F_3 can offer technical support for the approximated skyline index on *CH*.

A. F_1 Index

This section introduces $F_1(\alpha, G)$=$\alpha \otimes \Lambda^\circ$ as the baseline. When α=1, $F_1(1, G)$ reduces to *FHL*, yielding the exact index $1 \otimes \Lambda^\circ$=Λ°. As α increases, the likelihood of pruning more paths in Λ° rises, thus resulting in a smaller $\alpha \otimes \Lambda^\circ$ subset of Λ°. We first define the label in $\alpha \otimes \Lambda^\circ$ as follows:

Definition 12 (F_1 **label**). *In Λ°, each label $L_{\alpha_k}(v, a_i)$ is assigned an approximation ratio α_k<α, such that all the skyline paths in $L_{\alpha_k}(v, a_i)$ from v to their ancestor a_i cannot α_k-dominate each other, $\forall v \in V$.*

Then, we can establish the following lemma:

Lemma 4. *$L_{\alpha_k}(v, a_i)$ in Λ° is inadequate if it cannot encompass all the α_k-skyline paths from v to a_i, i.e., if $L_{\alpha_k}(v, a_i)$ adopts a larger approximation ratio than α_k. Conversely, $L_{\alpha_k}(v, a_i)$ is not the smallest if it includes additional $\alpha_{k'}$-skyline paths, which can be α_k-dominated, where $\alpha_{k'}$<α_k.*

Example 6. *When α=1.5, consider a label l=$\{(3,5),(6,2)\}$. The label l is not the smallest if l=$\{(2,6),(3,5),(6,2)\}$, as $(2, 6)$ can be α-dominated. Furthermore, l=$\{(6,2)\}$ is inadequate.*

Theorem 5 (**Approximate Label Error**). *Pruning each label in Λ° by α will result in inaccuracies in the query $q(s, t, C, \alpha)$.*

Proof. This can be illustrated through the following example: Given $L(v, w)$=$\{(4,4),(3,6)\}$, $L(w, u)$=$\{(5,6),(3,8)\}$, and a unique

cutting vertex w from v to u, we approximate labels for a query $q(u, v, C$=15, α=1.35).This updates $L(v, w)$ to $\{(4,4)\}$ and leaves $L(w, u)$ unchanged, resulting in paths $(9, 10)$ and $(7, 12)$. $(7, 12)$ is α-dominated, leaving $(9, 10)$ as the result of q. However, the exact sequence $\langle u, w, v\rangle$ yields skyline paths $\{(9,10),(7,12),(6,14)\}$. Removing α-dominated $(7, 12)$ and comparing $(9, 10)$ to $(6, 14)$, we find $(6, 14)$ is correct. Thus, directly assigning α to $L(v, w)$ is erroneous. □

Correctness of F_1. Each label $L_{\alpha_k}(v, a_i)$ must: (1) be minimum; and (2) answer all α-*CSP* queries accurately. Condition 1) is straightforwardly demonstrated as F_1 directly assigns an α_k to each exact label of Λ° such that each label cannot α_k-dominate any other. As for condition 2), for any origin-destination (OD) pair (s, t), $\alpha_0 \otimes \Lambda^\circ$ must support a querying method capable of finding all the α_0-skyline paths $\mathcal{P}_{\alpha_0}(s, t)$. This ensures that the α-*CSP* query from s to t with a variable C can be correctly answered.

In α-SPC_1, $\mathcal{P}_{\alpha_1}(s, h_i) \oplus \mathcal{P}_{\alpha_2}(h_i, t)$ must accurately compute $\mathcal{P}_{\alpha_0}(s, h_i, t)$ for $\forall h_i \in H'$. To achieve condition (2), the concatenation of all α_1-skyline paths and α_2-skyline paths must cover all α_0-skyline paths. In other words, the correctness of $\mathcal{P}_{\alpha_0}$ is contingent on α_0, α_1, and α_2. Consequently, we arrive at the following theorem.

Theorem 6 (Concatenate Relation). *α-SPC_1 correctly calculates $\mathcal{P}_{\alpha_0}(s, h_i, t)$ as $\mathcal{P}_{\alpha_1}(s, h_i) \oplus \mathcal{P}_{\alpha_2}(h_i, t)$ if $\alpha_1\alpha_2$=α_0.*

Proof. Due to the space limitation, we include the complete proof in our full version [28]. The proof commences with all-pair concatenations of paths from the α_1-skyline and α_2-skyline sets, thereby approximating α_1 and α_2 values by identifying the smallest gaps between path weights. Post-concatenation, we detect the largest weight ratio between paths, denoted as $w(p_2^2)/w(p_1^1)$, which is bounded by the product of α_1 and α_2. Utilizing a contradiction method, we affirm that this ratio cannot be less than α_0 without contradicting initial assumptions. We thus infer that α_0 is bound by this ratio and the product of α_1 and α_2, confirming the correctness of the concatenation operation for α_0=$\alpha_1\alpha_2$. □

The Operator $\otimes$. As outlined in Section II, the forest structure of α-*FHL* necessitates varying concatenation patterns. For a query (s, t, C, α), if the *OD* pair (s, t) resides within the same tree, α-SPC is executed once. Conversely, if $X_s \in T_1$ and $X_t \in T_2$, α-SPC is performed three times. Consequently, the operator $\otimes$ on each label in Λ° must facilitate accurate α-*CSP* query processing, even for the longest concatenation pattern. The operator $\otimes$ in F_1 distributes an approximation ratio evenly among each small tree and the boundary tree, i.e., $\alpha \otimes \Lambda^\circ$ approximates each label in Λ° by $\alpha^{1/6}$, ensuring that each label $L_{\alpha^{1/6}}(v, a_i)$ is calculated and each tree receives an allocation of $\alpha^{1/3}$.

Processing of the F_1 **Query**. The longest concatenation pattern warrants further discussion. Assuming sets H_1 and H_2 comprise all boundary vertices of T_1 and T_2 respectively, the following concatenations are considered: 1) $\mathcal{P}_{\alpha^{1/3}}(s, h_i)$ for $\forall h_i \in H_1$ in T_1; 2) $\mathcal{P}_{\alpha^{2/3}}(s, h'j)$ for $\forall h'j \in H_2$, calculated by $\mathcal{P}_{\alpha^{1/3}}(s, h_i)$ and $\mathcal{P}_{\alpha^{1/3}}(h_i, h'j)$ from T_1 to T_b; and 3) $\mathcal{P}_{\alpha}(s, t)$, computed by $\mathcal{P}_{\alpha^{2/3}}(s, h'j)$ and $\mathcal{P}_{\alpha^{1/3}}(h'_j, t)$ from T_b to T_2.

Analysis of F_1. In a real road network, the pruning power of F_1 on each label proves significant because α is not overly diluted. However, F_1 incurs considerable expense during index construction due to the prerequisite construction of an Λ° prior to approximation. Despite this, through the analysis of F_1, we identify potential issues relating to the allocation of α and elucidate the relationship between the number of concatenations and the allocation of α. This analysis is integral to the design of F_2 and F_3.

V. F_2 Index

This section details $F_2(\alpha, G)$=$f(\alpha, g(1, G))$, the pruning labeling index during the *approximate label assignment* phase. F_2 introduces three methods for approximating the labeling index: 1)F_2-E approximates each label based on the tree structure; 2)F_2-θ establishes a threshold θ and approximates labels exceeding this threshold; 3)F_2-R reuses wasted local ratios in subsequent concatenations.

A. Approximate Label Assignment

Given the forest structure of α-*FHL*, we acknowledge that each tree accommodates a portion of α. For the various approximation ratios, we provide the following definitions:

Definition 13 (Approximate Global Ratio). *Each smaller tree T_i possesses an approximate global ratio, denoted by α_i°. The approximate global ratio for the boundary tree is represented as α_b°. Hence, for any two values α_i° and α_j°, we have $\alpha_i^\circ\alpha_j^\circ\alpha_b^\circ$=$\alpha$.*

Definition 14 (Approximate Local Ratio). *Within tree T_k, each label $L(v, a_i)$ is assigned an approximate local ratio $\alpha_k(v, a_i)$, where $\alpha_k(v, a_i) \in (1, \alpha_k^\circ]$.*

We refer to $\alpha_k(v, a_i)$=$\alpha(v, a_i)$, α_i°=α° if tree is unspecified.

In $F_2(\alpha, G)$=$f(\alpha, g(1, G))$, F_2 initially constructs the same skyline tree decomposition as in *FHL*. For $\forall v \in V$, it extracts a set of exact skyline shortcuts $P_s(v, u)$, where $u \in N(v)$. Subsequently, F_2 performs a top-to-bottom *Approximate Label Assignment* to compute $\alpha(v, a_i)$-skyline paths for each label $L(v, a_i)$. This leads to the following equation:

$$L(v, a_i)=\mathcal{P}_{\alpha(v,a_i)}=\alpha\text{-}SPC_n(\bigcup\nolimits_{u\in N(v)} P_s(v, u) \oplus L(u, a_i)) \quad (5)$$

where a_i and u are ancestors of v. Additionally, u is a neighbour of v recorded during the graph contraction in *Skyline Tree Decomposition*. Therefore, $L(u, a_i)$=$\mathcal{P}_{\alpha(u,a_i)}(u, a_i)$ is computed before $L(v, a_i)$. The operator α-SPC_n serves to prune the vertices in $N(v)$ and validate the $\alpha(v, a_i)$-skyline paths from the union.

As per Equation 5, the paths in $L(v, u)$ may partake in the concatenations for distinct labels as a set of subpaths. This number is tied to traversed hops from v to u in the tree. Moreover, each concatenation of a path can be seen as an approximation operation on it. Consequently, we propose the following theorem:

Theorem 7 (**Concatenation and Approximation**). *The number of concatenations a path in $L(v,u)$ undergoes equals the number of times the path is approximated minus one.*

Proof. Theorem 6 provides the basis for our proof. Suppose $L(v,a_2)$ results from the concatenation of $L(v,a_1)$ and another label L_1=$\mathcal{P}\alpha_1$. Hence, we have $\alpha(v,a_i)\alpha_1$=$\alpha(v,a_2)$. This means that the paths in $L(v,a_1)$ transform into a set of sub-paths $Psub$ in $L(v,a_2)$. To obtain $\alpha(v,a_2)$, P_{sub} is approximated by α_1. When $L(v,a_2)$ concatenates with a label L_2=$\mathcal{P}\alpha_2$, the paths in $Psub$ are further approximated by $\alpha(v,a_i)\alpha_1\alpha_2$. As a result, if part of the paths in P_{sub} undergoes n concatenations and are saved as index, they are approximated by $\alpha(v,a_i)\alpha_1\alpha_2...\alpha_n$. □

With Theorem 7, we derive the following corollary.

Corollary 8 (**Approximate Times Towards Ancestors**). *Given a label $L(v,a_i)$, the maximum number of approximations a path $p \in L(v,a_i)$ undergoes is equivalent to the count of ancestors from v to a_i.*

Therefore, for $p \in \mathcal{P}_{\alpha_\circ}$, assume p is composed of subpaths $\langle p_1,\ldots,p_k\rangle$, we can then express this relationship with the following Inequality:

$$w(p_1)\prod_{i=1}^{k}\alpha_i + w(p_2)\prod_{i=2}^{k}\alpha_i + ... + w(p_k)\prod_{i=k}^{k}\alpha_i \le \alpha_\circ w(p) \quad (6)$$

Here, k denotes the maximum number of approximations.

F_2 is designed in adherence to Inequality 6. When we concatenate a label $L(v,a_i)$, it is crucial that each concatenated path in $L(v,a_i)$ satisfies Inequality 6. Failing to meet this inequality could result in an insufficient labelling index, as the sub-paths on the left side of Inequality 6 could be overly approximated, leading to the pruning of several essential skyline paths. Consequently, F_2 assigns an approximation ratio to each label, ensuring that the results of its concatenation align with Inequality 6.

B. F_2-E

Our initial method allocates an identical approximate local ratio for each hop. For a vertex v where X_v has n ancestors and X_{a_1} is the lowest ancestor of X_v, we set $\alpha(v,a_1)$=$\alpha^{\frac{1}{n}}$. The following theorem encapsulates this process.

Theorem 9 (**Even Approximate Local Ratio**). *Given label $L(v,a_i)$, where the tree height is $\mathcal{H}$, and X_v has k ancestors from X_v to X_{a_i} (including X_v), we allocate $\alpha^{\frac{k}{\mathcal{H}-1}}$ to $L(v,a_i)$.*

F_2-E Indexing. As shown in Algorithm 2, the input is T_G, constructed during the *skyline tree decomposition*. We calculate the labels for each tree node from the root (line 1). For each tree node X_v, we compute labels from v to each ancestor vertex u (lines 3-7). If the number of ancestors between v and u is k, then $\alpha(v,u)$=$\alpha^{\frac{k}{\mathcal{H}-1}}$. We then employ $\alpha(v,u)$ to approximate the concatenated result of $L(v,u)$ in the operator α-SPC_n.

Algorithm 2: Approximate Label Assignment

Input: Tree T_G with shortcuts by index contraction
Output: α-*FHL* Labeling L

1 $X_v \leftarrow$ Tree Root, $Q.insert(X_v)$ **while** $X_v \leftarrow Q.pop()$ **do**
2 $\mathcal{H} \leftarrow$ tree height;
3 **for** $u \in \{u | X_u$ *is ancestor of* $X_v\}$ **do**
4 $k \leftarrow$ the number of ancestors between v to u;
5 $\alpha(v,u)$=$\alpha^{\frac{k}{\mathcal{H}-1}}$;
6 $L(v,u) \leftarrow \alpha$-$SPC_n((L(v,w) \oplus L(w,u)), \forall w \in X_v \leftarrow \alpha_v$;
7 $L(v) \leftarrow (u, L(v,u))$;
8 $Q.insert(X_u.children)$, $L \leftarrow L \cup L(v)$;

Analysis of F_2-E. The determination of each approximate local ratio hinges upon the established tree structure, thereby linking the approximate power to the tree height. For a label $L(v,u)$, should X_v and X_u be proximate within a tree of substantial height $\mathcal{H}$, $\alpha(v,u)$ will approach 1. Conversely, F_2-E apportions excessive approximation ratios to labels at the base of the tree when k closely approximates n. This can result in smaller-sized labels wasting pruning power. We thus propose two optimized versions based on F_2-E in Section V-C.

C. F_2-θ and F_2-R

1) F_2-θ: In F_2-θ, we introduce a threshold θ during *Approximate Label Assignment*. For a label $L(v,u)$, if its size exceeds θ, an approximate local ratio is allocated. If not, we retain its concatenated result.

$|L(v,u)|$ can be confirmed after an exact concatenation. Nonetheless, this approach necessitates computing all exact skyline paths and subsequently approximating labels that satisfy θ, which can be extremely time-consuming.

Instead of calculating an exact $|L(v,u)|$, F_2-θ determines the number of concatenated paths N using the sizes $|L(u,h_i)|$ and $|L(h_i,v)|$. It then derives a weight upper bound w_{max} and a weight lower bound w_{min} prior to concatenation. Hence, θ=$\log_{\alpha(v,u)}(w_{max}/w_{min})$, where $\alpha(v,u)$ is the approximate local ratio of $L(v,u)$. We prune $L(v,u)$ if its $N > \log_{\alpha(v,u)}(w_{max}/w_{min})$.

*2) F_2-**R**:* F_2-R serves as an optimized variant of F_2-E. Assuming w is a tree root, the principal idea of F_2-R is to reclaim the pruning power of a label from u to the tree root, then repurpose the reclaimed ratio to compute $L(v,u)$. The reclaimed ratio is denoted as $\alpha.rec$. Accordingly, we propose the following equation.

$$\alpha(v,a_i)=(\alpha^\circ)^{\delta/\mathcal{H}-1}\cdot\alpha.rec(a_i,root) \quad (7)$$

where δ=$\mathcal{H}-\mathcal{H}_{a_i}$. To compute local ratio $\alpha(v,a_i)$, we derive $\alpha.rec$ via a recycling operator. We emphasize that the recycle is only activated on the labels that include the root.

Example 7 (**Local Ratio Recycling**). *In Figure 1(b), we take tree nodes X_1, X_2, X_3, and X_4 as an example. Suppose that we recycle $\alpha.rec(v_4,v_1)$ from $\alpha_\circ^{\frac{1}{7}}$ from $L(v_4,v_1)$, then $\alpha(v_4,v_1)$=$\alpha^{\frac{1}{7}}/\alpha.rec(v_{11},v_1)$. When we compute $L(v_3)$, we create two labels $L(v_3,v_1)$ and $L(v_3,v_4)$. We obtain $L(v_3,v_4)$=$(\alpha^\circ)^{\frac{1}{7}\cdot\alpha.rec(v_4,v_1)}$. $\alpha(v_3,v_1)$ is still $\alpha_\circ^{\frac{2}{7}}$. Then, we*

recycle $\alpha.rec(v_3, v_1)$. *For computing* $L(v_2)$, $\alpha(v_2, v_4)$ *is* $\alpha_\circ^{\frac{2}{7}} \cdot \alpha.rec(v_4, v_1)$ *and* $L(v_2, v_3)$ *is* $\alpha_\circ^{\frac{1}{7}} \cdot \alpha.rec(v_3, v_1)$. $\alpha(v_2, v_1)$ *is* $\alpha_\circ^{\frac{3}{7}}$ *and we execute the recycle operator on it.*

Before we discuss the recycling operator, we define the lower bound of dominance in the following.

Definition 15 (Lower-bound of Domination). *Given a set of paths in cost-increasing order* $\{p_1, p_2, ..., p_k, ..., p_n\}$, *and an approximation ratio* α_i, *if* p_k *is* α_i*-dominated by* p_{k-1}, d_k=$w(p_{k-1})/w(p_k)$ *is a lower bound of* p_k. *Thus,* d_k *is the lowest approximation ratio to dominate* p_k.

Recycle Operator is integrated into the α-SPC_1 of each label $L(v, u)$. It calculates the lower bound d_k for each dominated path p_k from v to u and compares d_k with $\alpha(v, u)$. If d_k=$\alpha(v, u)$ is found, we do not recycle $\alpha(v, u)$. Otherwise, we select the maximum d_k denoted by d_{max}, and set $\alpha.rec(v, u)$=$\alpha(v, u)/d_{max}$.

Example 8. *In Figure 2(c),* α-SPC_1 *produces six paths via* h_i *with a cost-increasing order. The first* α_i*-dominated path is* p_2, *we compute* d_2=$w(p_1)/w(p_2)$. *Then, we get rid of* p_2 *and concatenate* p_3. *However,* p_3 *cannot be* α_i*-dominated by* p_1, *so we do not compute* d_3. *The second* α_i*-dominated path is* p_5, *and* d_5=$w(p_4)/w(p_5)$. *Subsequently, if* d_5<d_2<α_i, *we can recycle* α_i/d_2 *and the concatenate results are not affected.*

VI. F_3 Index

In this section, we discuss $F_3(\alpha, G)$=$f(1, g(\alpha, G))$ on the *Skyline Tree Decomposition* phase. For every vertex $v \in V$, F_3 approximates the skyline shortcuts $P_s(v, u)$ from v to each neighbour u in the contracted graph, with the restriction that the pruning power of each shortcut cannot exceed α°.

During the contraction process, shortcuts between neighbours u and w fall into three distinct situations: (1)Both (v, u) and (v, w) are original edges of G; (2)(v, u) is an original edge of G, but (v, w) is a shortcut in the contracted graph; and (3)Both (v, u) and (v, w) are shortcuts.

For situation one, a shortcut can be straightforwardly approximated with a local ratio. However, situations two and three pose challenges due to inequality 6. Specifically, $\alpha(v, u)\alpha(v, w) \leq \alpha^\circ$ must be considered for approximating the shortcut of a pair (u, w).

In the contraction process, we aim to establish efficient shortcuts with approximated weights based on the original edge weights w_i. This process relies on a designated contraction order. For each vertex contraction, shortcuts are created and their approximated weights are determined by multiplying the original edge weight with a predefined approximation ratio α_i. When subsequent vertex contractions introduce potential shortcuts between the same pair of nodes, we must update these paths by evaluating their approximated dominance. This assessment requires aligning shortcuts under the same approximation ratio. In cases where a newly introduced shortcut outperforms the existing one in terms of its approximated weight, the approximation ratio of the path is updated. In the scenarios, it is essential to ensure that the updated ratio does not exceed the global ratio α°.

Example 9. *Figure 3-(a) illustrates the approximation of shortcuts in a partition of* G°. *We contract vertices in the order* v_5, v_2, v_3, v_4, v_1. *Initially, we contract* v_5*(Figure 3-(b)), resulting in an approximation of shortcut weight* $\alpha_1 w(v_1, v_4)$=$\alpha_1(w_1$+$w_2)$. *After contracting* v_2*(Figure 3-(c)), the generated shortcuts yield distinct approximate weights,* $\alpha_2 w(v_1, v_3)$=$\alpha_2(w_4, w_7)$, $\alpha_3 w(v_3, v_4)$=$\alpha_3(w_3$+$w_7)$, *and* $\alpha_4 w(v_1, v_4)$=$\alpha_4(w_3$+$w_4)$. *We allocate* $\alpha_1(v_1, v_4)$ *when contracting* v_5, *so shortcuts from* v_1 *to* v_4 *are updated by verifying the approximate dominance. This requires the alignment of shortcuts with the same ratio, hence* α_1=α_4. *Consequently, we set* α_1=α_2=α_3=α_4. *The next contraction* v_3 *yields* $\alpha_5 w(v_1, v_4)$=$\alpha_5(\alpha_2 w(v_1, v_3)$+$\alpha_3 w(v_3, v_4))$. *This new shortcut is then combined with previous ones between* v_1 *and* v_4, *selecting the path with the largest ratio. This results in the update of local ratio of* (v_1, v_4) *to* $\alpha_5\alpha_2$, *where* $\alpha_5\alpha_2 > \alpha_1$=$\alpha_2$. *Lastly, we ensure* $\alpha_5\alpha_2 \leq \alpha$.

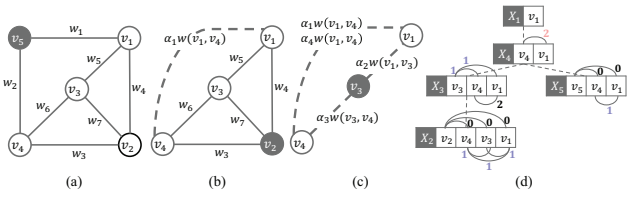


Fig. 3. Example of F_3 indexing

Suppose we allocate local ratios evenly. In this case, we can determine the local ratio once we establish the greatest number of contractions necessary for computing a shortcut.

In light of this, F_3 initially creates a tree structure for each subgraph but skips the computation on shortcuts. It leads to a constructed tree only encompassing tree nodes with neighbourhood relations. Moving from the bottom to the top of the tree, F_3 computes the number of contractions for any pair (v, u) in every tree node as extra information. Specifically, we denote the number of contractions of any shortcut (v, u) involved as $\lambda(v, u)$. By applying Theorem 7, we can deduce that $\lambda(v, u)$ equals the highest approximation times of (v, u). For example, in Figure 3-(d), $\lambda(v_1, v_4)$=2, as the computation of a shortcut of (v_1, v_4) go through the rate α_2 and α_5. The computation of $\lambda(v, u)$ follows a cumulative process within the tree structure constructed, as discussed in the following.

Theorem 10 (Cumulative Computation on λ). *For a tree node* X_v *consists of* $\{w, u\}$, *if* $\lambda(v, w)$=m *and* $\lambda(v, u)$=n, *then* $\lambda(w, u)$=$\max\{m, n\}$+1.

Proof. The proof can be derived from Theorem 7, by regarding each shortcut contraction as a concatenation operation. □

As shown in Figure 3-(d), which is the tree structure of Figure 3-(a), our computation starts from the leaves, thus we begin with X_2. The neighbours of v_2, namely v_4, v_3,

and v_1, have no approximations for (v_2, v_4), (v_2, v_3), and (v_2, v_1), as they are the original edges. Hence, we assign their λ values as zero. According to Theorem 10, the other pair of neighbours can be calculated as 1, such as $\lambda(v_4, v_3)$, $\lambda(v_3, v_1)$, and $\lambda(v_4, v_1)$. Next, we move to the other leaf, X_5, and assign $\lambda(v_5, v_4)$ and $\lambda(v_5, v_1)$ as zero, resulting in $\lambda(v_4, v_1)$=1. Upon visiting the parent of X_5, we label $\lambda(v_4, v_1)$=1 in X_4. Similarly, while visiting the parent of X_2, we assign $\lambda(v_3, v_4)$ and $\lambda(v_3, v_1)$ as 1 based on the values in X_2. Consequently, $\lambda(v_4, v_1)$ in X_3 is calculated as 2. When travelling from X_3 to X_4, we update $\lambda(v_4, v_1)$ in X_4 from 1 to 2.

Next, we allocate an $\alpha(v, u)$ to $P_c(u, v)$ in the following:

Theorem 11 (**Local Ratio Allocation with λ**). *Assume that the largest λ in a tree T_G is n, and $\lambda(u, w)$=k for a set $P_c(u, w)$ of concatenated paths in X_v, we can allocate $\alpha^{\frac{k}{n}}$ to $P_c(u, w)$ for the dominance operation.*

Example 10. *We can assign $\alpha^{\frac{1}{2}}$ to $P(v_4, v_1)$ of X_5. As a result, the pruning power of $P(v_4, v_1)$ in X_4 increases to α because $\lambda(v_4, v_1)$ is updated to 2.*

Analysis of F_3. For each partition in the graph, a tree structure is created to track the number of contractions between each pair. The maximal number of contractions needed to create a shortcut determines the distribution of local ratios. During *skyline label assignment*, the approximate power may be diluted as no further consideration is given to the approximate ratio. Nonetheless, F_3 not only provides a more comprehensive index than F_2 but also offers a viable solution for approximating the *CH*-based index.

VII. Experiments

We implemented all algorithms in C++ with O3 optimizations and conducted in a 64-bit Ubuntu 18.04.3 LTS with two 8-core Intel Xeon CPU E5-2690 2.9GHz and 186GB RAM.

Datasets. All tests are carried out on five real road networks obtained from the 9^{th} *DIMACS* Implementation Challenge [29]. We use the length of the road as weight w and the travel time t as the cost. We list the information from the networks as follows: ***NY*** is a dense grid-like urban area with 264,246 vertices, 733,846 edges and several partitions connected by bridges; ***BAY*** has a doughnut shape around *San Francisco Bay* with 321,270 vertices, 800,172 edges; ***COL*** has an uneven vertex distribution (dense around Denver but sparse elsewhere) with 435,666 vertices and 1,057,066 edges; ***FLA*** is Florida with 1,070,376 vertices and 2,712,798 edges; ***LKS*** is the largest network Great Lakes, which is about ten times larger than ***NY***, with 2,758,119 vertices and 6,885,658 edges.

Query Set. For each dataset, we randomly generate five sets of *OD* pairs Q_1 to Q_5, each with 1000 queries. Specifically, we first estimate the diameter d_{max}, which is the longest length of all shortest paths in the graph [30]. Then each Q_i represents a category of *OD* pairs that fall in the distance range $[d_{max}/2^{6-i}, d_{max}/2^{5-i}]$. Afterwards, we assign the query constraints C to these *OD* pairs. First, we calculate the cost range $[C_{min}, C_{max}]$ for each *OD*. This is because if C<C_{min}, the optimal result cannot be found, then the query is invalid; and if C>C_{max}, the optimal result is the path with the shortest distance. Specifically, each *OD* pair has a constraint: C=rC_{max}+$(1-r)C_{min}$, with $r \in (0, 1)$. When r is small, C is closer to the minimum cost; as r increases, C is closer to the maximum cost.

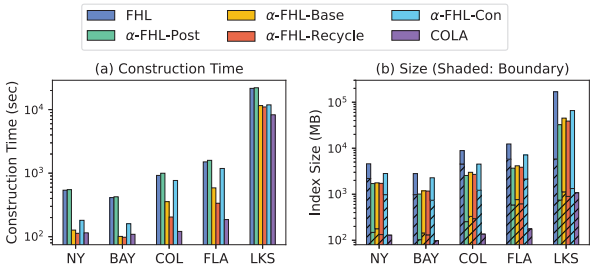


Fig. 4. Index Construction Performance

Algorithms. We compare our methods with *Sky-Dijkstra* [31] and *COLA* [25]. As our methods adapt different approximation strategies to *FHL*, we list them as follows: ***α-FHL*-Post** is F_1 index approximated on the exact *FHL*; ***α-FHL*-Base** is F_2-E, which assign a_i evenly; ***α-FHL*-Recycle** is the optimized α-*FHL*, including F_2-R and F_2-θ; ***α-FHL*-Con** performs F_3 in index contraction.

A. Index Size and Construction Time

Figure 4-(b) illustrates the index sizes of *FHL*,α-*FHL*s and *COLA* in five different road networks, when we set α=1.1. The index of *FHL* is exacted. Therefore, we compare it with α-*FHL* to show the effectiveness of α-*FHL* in pruning the index. For each bar, the shaded area shows the index size of the boundary tree, and the non-shaded area is the size of the small trees. We can observe that the best pruning result is α-*FHL*-Post, because it prunes the index based on the exact *FHL* index. The index size of α-*FHL*-Recycle is very close to α-*FHL*-Post. Moreover, all methods for approximate label assignments, including α-*FHL*-Base and α-*FHL*-Recycle, provide better results than α-*FHL*-Con, which approximates the index in index contraction. This distinction arises because the number of intermediate skyline paths (i.e., skyline shortcuts) created during index contraction is significantly lower than those generated in label assignment. Specifically, the pruned shortcuts typically cover short distances. As a result, the retained shortcuts frequently generate exact skyline paths without pruning through concatenations during the label assignment phase. Nevertheless, it offers a more complete index that reflects the lower bound of query error for α. Significantly, the pruning within the boundary tree remains most impactful for α-*FHL*s. In the case of *COLA*, it boasts the smallest index, as it mainly includes the skyline paths between the boundary vertices.

Figure 4-(a) shows the index construction time of *FHL*,α-*FHL*s and *COLA* in five different road networks when we set α=1.1. α-*FHL*-Post takes the longest time. On the contrary, the indexing time of α-*FHL*-Recycle is the shortest among α-

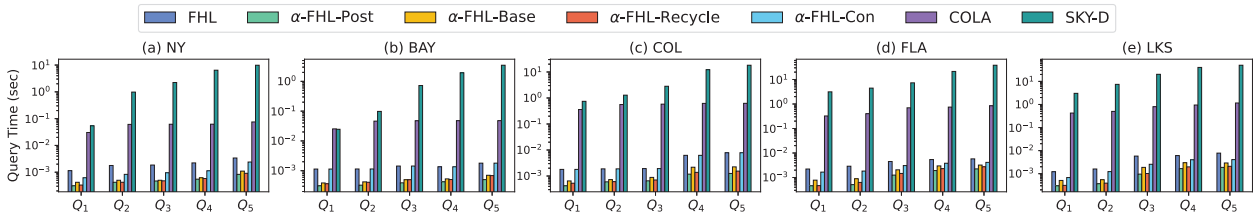


Fig. 5. Query Time

*FHL*s. For *COLA*, only the skyline paths between the boundary vertices are considered. Thus, it has faster pre-processing.

Among the α-*FHL*s, the α-*FHL*-Recycle is the fastest to construct and is very close to the best approximate result. Although α-*FHL*-Recycle has a larger index size and longer indexing time than *COLA*, α-*FHL*-Recycle supports faster query processing, and index construction is still practical. Consider that α-*FHL*-Recycle can avoid slow online search in query processing, and its indexing cost is worth paying for.

For the largest graph *LKS*, the construction time of all algorithms increases significantly compared to the previous four graphs, with *FHL* being particularly affected. Tree decomposition-based methods incur the greatest indexing time on the boundary graph. In our experiments, we aimed to minimize the number of boundary vertices for each graph (ranging from 2452 in *NY* to 7444 in *LKS*) to limit the boundary edges. However, due to the inherent structure of *LKS*, we are constrained to 72 partitions (similar to *NY*) which resulted in numerous long-distance skyline paths among the boundary vertices in each partition. Conversely, further partitioning of *LKS* would not yield improvements; due to its structure, it would increase the number of boundary vertices, and consequently, the edge count, hindering the performance of indexing. A similar decline in performance is observed with *COLA*. Nonetheless, α-*FHL*-Recycle managed to reduce the indexing time and size of *FHL* markedly, demonstrating the efficacy and effectiveness of our approximate methods.

B. Query Efficiency

Figure 5 presents the query processing time for the comparative methods when α=1.1 and r=0.5. We plot the results on four graphs from Q_1 to Q_5.

First, our α-*FHL*s outperform all other methods by several orders of magnitude. Especially, α-*FHL*-Recycle always answers queries within 1 millisecond, regardless of the query distance in *NY* and *BAY*. Compared to online search methods, the query distance has less effect on the performance of our methods due to the hop labelling-based index. The query time has slightly increased along with the distance because queries with larger distances may cross the tress and then experience multiple concatenations. Furthermore, among the α-*FHL*s, α-*FHL* has the best performance. This is because the smallest index size reduces the number of concatenated paths. Therefore, it saves the cost of the dominance operation. Subsequently, the performance of α-*FHL*-Recycle is very close to that of α-*FHL*-Post.

Moreover, our methods avoid the online search and are several orders of magnitude faster than *COLA* on querying. This is because *COLA* adopts *Sky-Dijkstra* search on the partitions of start and destination. It takes $O(wV(V \log V+E))$ time, where w is the longest shortest path in a graph partition. However, α-*FHL* takes $O(mnH)$ for query answering, where m and n are the start and destination label sizes, which is much smaller than V. H is the intermediate hop size.

Combine with the performance of α-*FHL*-Recycle in index construction. We determine that α-*FHL*-Recycle is the most practical one among α-*FHL*s, because it has the fastest index contraction time. Moreover, the index size and query processing are similar to those of α-*FHL*-Post. Therefore, we will test α-*FHL*-Recycle with different α in Section VII-C.

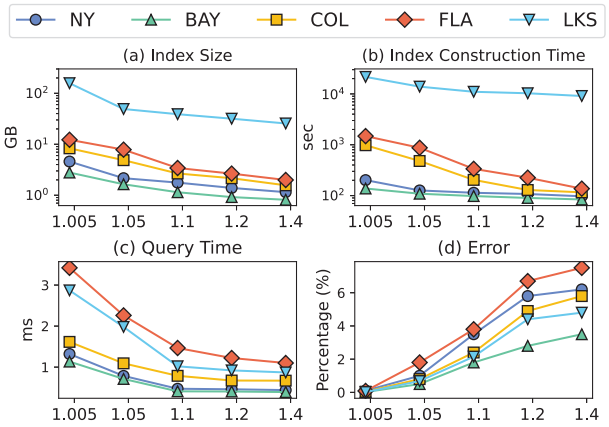


Fig. 6. α-FHL-Recycle Experiment

C. α-FHL with Different α

In Figure 6, we test the index size, constructing time, query execution time, and query accuracy by varying α. We plot the results for five different datasets.

Figure 6-(a) shows the index space of α-*FHL*-Recycle on all the five graphs. We observe that the index size decreases when we change α from 1.005 to 1.4. The index sizes are pruned by more than half when $\alpha \geq 1.1$. The improvement in index construction time is also significant, as shown in Figure 6-(b). This is because a larger α results in more pruned paths during the label assignment phase. Since α-FHL computes the labels from top to bottom of the tree, the approximated labels will be used for the computation of the deeper tree nodes. Increasing

α will save time for the concatenation of each label, and thus reduce the indexing time.

In Figure 6-(c), we can observe that the query processing time is slightly reduced when we increase α. For explanation, α-*FHL*-Recycle answers queries by concatenating two labels. The increased pruning power results in the reduction of the labels for concatenating. Therefore, there are fewer concatenated paths satisfying our constraint on the attribute cost, thus saving the dominance verification time, especially for queries from different trees.

In the end, we randomly generate 500 queries to test the accuracy of our method. For each query, we compute its relative error by $w(p')/w(p) - 1$, where $w(p)$ is the weight of the exact result obtained by *FHL*, and $w(p')$ is the weight of α-*CSP*. As shown in Figure 6-(d), a higher α results in a larger relative error. The good balance of α is from 1.1 to 1.2 for α-*FHL*-Recycle.

VIII. Related Works

Four main *CSP* algorithm types exist, categorized by their use of exact/approximate and index-free/index methods.

Exact Index-Free CSP. The first study of the Constrained Shortest Path (CSP) problem, by [32], used dynamic programming, while [33] converted it into an integer linear programming problem. However, these weren't scalable for large networks. Practical solutions, such as *Skyline Dijkstra* and *k-Path* [34], work by enforcing cost constraints and examining top-k paths respectively. Improvements include [35]'s Pulse, which improved *k-Path* with constraint pruning, and *cKSP* [36] and *eKSP* [37], which refined the search strategies.

Approximate Index-Free CSP The approximation allows the result length to be not longer than $(1+\alpha)$ times the optimal path. The first approximate solution is also proposed by [20] with a complexity of $O(|E|^2 \frac{|V|^2}{\alpha-1} \log \frac{|V|}{\alpha-1})$, and [38] improves it to $O(|V||E| \ (\log\log \frac{w_{max}}{w_{min}} + \frac{1}{\alpha-1}))$. *Lagrange relaxation* is applied by [33], [39], [40] to obtain the approximation result with $O(|E| \log^3 |E|)$ iterations of the pathfinding. As shown in [41] and [35], these approximate algorithms are even orders of magnitude slower than the *k-Path*-based exact algorithms. *CP-CSP* [31] approximates *Skyline Dijkstra* by distributing the approximation power $\sqrt[n]{\alpha}$ to the edges. However, this approximation ratio would decrease to almost 1 when the graph is large, so its performance is only slightly better than *Sky-Dijk*.

Exact Index-Based CSP. [23] utilizes *skyline paths* in its shortcuts but suffers from large index and slow queries. While *CSP-CH* limits the skyline paths in each shortcut, it still has performance issues. *FHL* [18], [24] and *FHL-Cube* [17] apply hop labelling techniques to *CSP* and *MCSP* but are limited by their large index size and construction time.

Approximate Index-Based CSP The only approximate index method is *COLA* [25], which partitions the graph into regions and precomputes approximate skyline paths between regions. Similar to *CP-CSP*, the approximate paths are also computed in the *Sky-Dijk* fashion. To avoid the diminishing approximation power $\sqrt[n]{|V|}$, it views α as a budget and concentrates it on the important vertices. A similar pruning technique is also applied in time-dependent pathfinding [4]. In this way, the approximation power is released, so it has a faster construction and smaller index.

Label Constrained CSP is another stream of *CSP* whose constrained dimension is discrete label instead of numeric values [42]–[45]. It is used in scenarios where certain kinds of roads should be avoided.

IX. Directed, Multi-Dimensions, and Dynamic

[Directed] This extension requires two changes: 1) Vertex v neighbours are distinguished as *in*-neighbours $N_i(v)$ and *out*-neighbours $N_o(v)$. During contraction, a directed pair (u_1, u_2) is formed by combining one $u_1 \in N_i(v)$ and one $u_2 \in N_o(v)$; 2) Rather than saving skyline paths between v and its neighbours in each $X(v)$, skylines from u_1 to v are marked as *in*-label, and those from v to u_2 as *out*-label, indicating direction towards v. Then for a query $q(s, t, C, \alpha)$, determining $X(LCA(s, t)$ is the same, while the concatenation requires *out*-label for s and *in*-label for t as hops.

[Multi-dimensional] Suppose a path set P_s comprising n-dimensional skyline paths, each holding weight w and n-1 costs [17], [18]. Our α emphasis on w ensures each cost comparison abides by the exact skyline path principle. This means the reliability of our method is contingent upon the approximation of w, guaranteeing the correct α-*FHL* outcomes in multi-dimensional queries.

[Dynamicity] It is non-trivial to design an incremental algorithm to update the skyline path index on tree decomposition. The straightforward way is to find each affected label and recompute them [46]–[50], but it would be time-consuming for the skyline paths. However, a more fine-grained approach is to locate the specific paths in each affected label, which will be investigated thoroughly in our recent future studies.

X. Conclusion

We present α-*FHL*, an efficient solution for α-*CSP* queries, using approximate skyline path concatenation with pruning techniques for index construction. Our analysis provides strategies for effective pruning with theoretical guarantees. Evaluations on real-life road networks show our methods outperform existing *CSP* approaches and reduce index construction time and size significantly.

Acknowledge

This work was partially supported by the Australian Research Council under Grant No. DP200103650 and LP180100018, Hong Kong Research Grants Council (grant# 16202722), NSFC #62202116, Guangzhou-HKUST(GZ) Joint Funding Scheme #2023A03J0135, Jiangsu Big Data Intelligent Lab Project #SDGC2228, and was partially conducted in the JC STEM Lab of Data Science Foundations funded by The Hong Kong Jockey Club Charities Trust.

REFERENCES

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[2] L. Li, M. Zhang, W. Hua, and X. Zhou, "Fast query decomposition for batch shortest path processing in road networks," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, 2020.

[3] M. Zhang, L. Li, W. Hua, and X. Zhou, "Stream processing of shortest path query in dynamic road networks," *IEEE Transactions on Knowledge and Data Engineering*, 2020.

[4] L. Li, S. Wang, and X. Zhou, "Time-dependent hop labeling on road network," in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*, April 2019, pp. 902–913.

[5] L. Li, S. Wang, and X. Zhou, "Fastest path query answering using time-dependent hop-labeling in road network," *IEEE Transactions on Knowledge and Data Engineering*, 2020.

[6] S. Wang, W. Lin, Y. Yang, X. Xiao, and S. Zhou, "Efficient route planning on public transportation networks: A labelling approach," in *Proceedings of the 2015 ACM SIGMOD international conference on Management of data*, 2015, pp. 967–982.

[7] H. Wu, J. Cheng, S. Huang, Y. Ke, Y. Lu, and Y. Xu, "Path problems in temporal graphs," *Proceedings of the VLDB Endowment*, vol. 7, no. 9, pp. 721–732, 2014.

[8] J. Li, C. Ni, D. He, L. Li, X. Xia, and X. Zhou, "Efficient knn query for moving objects on time-dependent road networks," *The VLDB Journal*, pp. 1–20, 2022.

[9] L. Li, W. Hua, X. Du, and X. Zhou, "Minimal on-road time route scheduling on time-dependent graphs," *PVLDB*, vol. 10, no. 11, pp. 1274–1285, 2017.

[10] L. Li, K. Zheng, S. Wang, W. Hua, and X. Zhou, "Go slow to go fast: minimal on-road time route scheduling with parking facilities using historical trajectory," *The VLDB Journal*, vol. 27, no. 3, pp. 321–345, 2018.

[11] J. D. Adler and P. B. Mirchandani, "Online routing and battery reservations for electric vehicles with swappable batteries," *Transportation Research Part B: Methodological*, vol. 70, pp. 285–302, 2014.

[12] Z. Luo, L. Li, M. Zhang, W. Hua, Y. Xu, and X. Zhou, "Diversified top-k route planning in road network," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 3199–3212, 2022.

[13] H. Liu, C. Jin, B. Yang, and A. Zhou, "Finding top-k shortest paths with diversity," *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 3, pp. 488–502, 2017.

[14] T. Chondrogiannis, P. Bouros, J. Gamper, U. Leser, and D. B. Blumenthal, "Finding k-shortest paths with limited overlap," *The VLDB Journal*, pp. 1–25, 2020.

[15] J. Li, X. Xiong, L. Li, D. He, C. Zong, and X. Zhou, "Finding top-k optimal routes with collective spatial keywords on road networks," in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2023.

[16] J. M. Jaffe, "Algorithms for finding paths with multiple constraints," *Networks*, vol. 14, no. 1, pp. 95–116, 1984.

[17] Z. Liu, L. Li, M. Zhang, W. Hua, and X. Zhou, "Fhl-cube: multi-constraint shortest path querying with flexible combination of constraints," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 3112–3125, 2022.

[18] Z. Liu, L. Li, M. Zhang, W. Hua, and X. Zhou, "Multi-constraint shortest path using forest hop labeling," *The VLDB Journal*, pp. 1–27, 2022.

[19] P. Van Mieghem and F. A. Kuipers, "On the complexity of qos routing," *Computer communications*, vol. 26, no. 4, pp. 376–387, 2003.

[20] P. Hansen, "Bicriterion path problems," in *Multiple criteria decision making theory and application*. Springer, 1980, pp. 109–127.

[21] Q. Gong, H. Cao, and P. Nagarkar, "Skyline queries constrained by multi-cost transportation networks," in *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 2019, pp. 926–937.

[22] H.-P. Kriegel, M. Renz, and M. Schubert, "Route skyline queries: A multi-preference path planning approach," in *IEEE International Conference on Data Engineering (ICDE)*. IEEE, 2010, pp. 261–272.

[23] S. Storandt, "Route planning for bicycles—exact constrained shortest paths made practical via contraction hierarchy," in *ICAPS*, 2012.

[24] Z. Liu, L. Li, M. Zhang, W. Hua, P. Chao, and X. Zhou, "Efficient constrained shortest path query answering with forest hop labeling," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 1763–1774.

[25] S. Wang, X. Xiao, Y. Yang, and W. Lin, "Effective indexing for approximate constrained shortest path queries on large road networks," *Proceedings of the VLDB Endowment*, vol. 10, no. 2, pp. 61–72, 2016.

[26] W. Li, M. Qiao, L. Qin, Y. Zhang, L. Chang, and X. Lin, "Scaling distance labeling on small-world networks," in *Proceedings of the 2019 ACM SIGMOD international conference on Management of data*, 2019, pp. 1060–1077.

[27] L. Chang, J. X. Yu, L. Qin, H. Cheng, and M. Qiao, "The exact distance to destination in undirected world," *VLDB Journal*, vol. 21, no. 6, pp. 869–888, 2012.

[28] "Complete version of this paper," https://www.dropbox.com/scl/fi/qe0bxffhlga5q8iutbr1w/Approximate_FHL-ICDE2024-Complete.pdf?rlkey=yassr4s3rju8zfjjmy1lu07ko&dl=0.

[29] "9th dimacs implementation challenge," http://www.diag.uniroma1.it/~challenge9/download.shtml.

[30] U. Meyer and P. Sanders, "δ-stepping: a parallelizable shortest path algorithm," *Journal of Algorithms*, vol. 49, no. 1, pp. 114–152, 2003.

[31] G. Tsaggouris and C. Zaroliagis, "Multiobjective optimization: Improved fptas for shortest paths and non-linear objectives with applications," *Theory of Computing Systems*, vol. 45, no. 1, pp. 162–186, 2009.

[32] H. C. Joksch, "The shortest route problem with constraints," *Journal of Mathematical analysis and applications*, vol. 14, no. 2, pp. 191–197, 1966.

[33] G. Y. Handler and I. Zang, "A dual algorithm for the constrained shortest path problem," *Networks*, vol. 10, no. 4, pp. 293–309, 1980.

[34] J. Y. Yen, "Finding the k shortest loopless paths in a network," *Management Science*, vol. 17, no. 11, pp. 712–716, 1971.

[35] L. Lozano and A. L. Medaglia, "On an exact method for the constrained shortest path problem," *Computers Operations Research*, vol. 40, no. 1, pp. 378 – 384, 2013.

[36] J. Gao, H. Qiu, X. Jiang, T. Wang, and D. Yang, "Fast top-k simple shortest paths discovery in graphs," in *Proceedings of the 19th ACM international conference on Information and knowledge management*, 2010, pp. 509–518.

[37] A. Sedeño-Noda and S. Alonso-Rodríguez, "An enhanced k-sp algorithm with pruning strategies to solve the constrained shortest path problem," *Applied Mathematics and Computation*, vol. 265, pp. 602 – 618, 2015.

[38] D. H. Lorenz and D. Raz, "A simple efficient approximation scheme for the restricted shortest path problem," *Operations Research Letters*, vol. 28, no. 5, pp. 213–219, 2001.

[39] A. Juttner, B. Szviatovski, I. Mécs, and Z. Rajkó, "Lagrange relaxation based method for the qos routing problem," in *Proceedings IEEE INFOCOM 2001. Conference on Computer Communications. Twentieth Annual Joint Conference of the IEEE Computer and Communications Society*, vol. 2. IEEE, 2001, pp. 859–868.

[40] W. M. Carlyle, J. O. Royset, and R. Kevin Wood, "Lagrangian relaxation and enumeration for solving constrained shortest-path problems," *Networks: An International Journal*, vol. 52, no. 4, pp. 256–270, 2008.

[41] F. Kuipers, A. Orda, D. Raz, and P. Van Mieghem, "A comparison of exact and ε-approximation algorithms for constrained routing," in *NETWORKING*. Springer, 2006, pp. 197–208.

[42] R. Jin, H. Hong, H. Wang, N. Ruan, and Y. Xiang, "Computing label-constraint reachability in graph databases," in *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, 2010, pp. 123–134.

[43] Y. Peng, Y. Zhang, X. Lin, L. Qin, and W. Zhang, "Answering billion-scale label-constrained reachability queries within microsecond," *Proceedings of the VLDB Endowment*, vol. 13, no. 6, pp. 812–825, 2020.

[44] U. Zhang, L. Yuan, W. Li, L. Qin, and Y. Zhang, "Efficient label-constrained shortest path queries on road networks: a tree decomposition approach," *Proceedings of the VLDB Endowment*, 2021.

[45] L. D. Valstar, G. H. Fletcher, and Y. Yoshida, "Landmark indexing for evaluation of label-constrained reachability queries," in *Proceedings of the 2017 ACM International Conference on Management of Data*, 2017, pp. 345–358.

[46] M. Zhang, L. Li, W. Hua, R. Mao, P. Chao, and X. Zhou, "Dynamic hub labeling for road networks," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 336–347.

[47] M. Zhang, L. Li, and X. Zhou, "An experimental evaluation and guideline for path finding in weighted dynamic network," *Proceedings of the VLDB Endowment*, vol. 14, no. 11, pp. 2127–2140, 2021.

[48] M. Zhang, L. Li, W. Hua, and X. Zhou, "Efficient 2-hop labeling maintenance in dynamic small-world networks," in *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 2021, pp. 133–144.

[49] M. Zhang, L. Li, G. Trajcevski, A. Zufle, and X. Zhou, "Parallel hub labeling maintenance with high efficiency in dynamic small-world networks," *IEEE Transactions on Knowledge and Data Engineering*, 2023.

[50] Y. Zhang and J. X. Yu, "Relative subboundedness of contraction hierarchy and hierarchical 2-hop index in dynamic road networks," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 1992–2005.